

Agnos Candidate Assignment - Development Planning Documentation

ตำแหน่ง: Back-end Developer | ระบบ: Hospital Middleware System

Tech Stack: Go (Golang), Gin Framework, PostgreSQL 16, Docker, Nginx

1. โครงสร้างโปรเจกต์ (Project Structure)

ระบบถูกออกแบบตามหลักการ Clean / Layered Architecture เพื่อแยกหน้าที่ความรับผิดชอบอย่างชัดเจน ง่ายต่อการทดสอบ และรองรับการขยายตัวในอนาคต:

```
hospital-middleware/
├── cmd/
│   └── server/
│       └── main.go          # จุดเริ่มต้นของระบบ (Entrypoint) & Graceful Shutdown
├── internal/
│   ├── client/
│   │   └── his/             # ตัวเชื่อมต่อระบบ HIS ภายนอก (Hospital A API Adapter)
│   │       ├── client.go    # HIS Interface & Client Factory
│   │       ├── hospital_a.go # Adapter เชื่อมต่อ Hospital A API
│   │       ├── hospital_a_test.go # Unit tests สำหรับ Hospital A Client
│   │       └── mock_his.go   # Mock Server สำหรับทดสอบ
│   └── config/
│       └── config.go        # ตัวจัดการโหลด Configuration / Environment Variables
├── domain/
│   ├── hospital.go         # Entity โรงพยาบาล
│   ├── staff.go            # Entity เจ้าหน้าที่ & DTOs
│   └── patient.go           # Entity ผู้ป่วย & Search Criteria DTOs
├── handler/
│   ├── health_handler.go    # Health check API Controller (/health)
│   └── staff_handler.go      # Staff API Controller (/staff/create, /staff/login)
├── staff_handler_test.go     # Unit tests ของ Staff Controller
├── patient_handler.go        # Patient API Controller (/patient/search)
├── patient_handler_test.go   # Unit tests ของ Patient Controller
├── middleware/
│   ├── auth_middleware.go    # JWT Authentication & Multi-Tenancy Scoper
│   ├── auth_middleware_test.go # Unit tests ของ Auth Middleware
│   └── logger_middleware.go   # Request Logging & CORS Middleware
├── repository/
└── interfaces.go             # Data Access Interfaces
```

```

├── postgres/
│   ├── db.go          # PostgreSQL Connection & Auto Migrations
│   ├── hospital_repo.go # Data Access สำหรับโรงพยาบาล
│   ├── staff_repo.go   # Data Access สำหรับเจ้าหน้าที่
│   └── patient_repo.go # Data Access สำหรับผู้ป่วย (Dynamic Search
Query)
├── service/
│   ├── interfaces.go   # Business Logic Interfaces
│   ├── staff_service.go # Business Logic สำหรับ Staff (Bcrypt, JWT)
│   ├── staff_service_test.go # Unit tests ของ Staff Service
│   ├── patient_service.go # Business Logic ค้นหาผู้ป่วย & Sync ข้อมูล HIS
│   └── patient_service_test.go # Unit tests ของ Patient Service
├── testutil/
│   └── mock_repos.go    # In-memory Mock Repositories สำหรับ Unit
Testing
├── pkg/
│   ├── response/
│   │   └── response.go  # Helper จัดรูปแบบ JSON Response กลาง
│   ├── token/
│   │   ├── jwt.go       # JWT Generator & Validator
│   │   └── jwt_test.go  # Unit tests ของ JWT Manager
├── migrations/
│   ├── 000001_init_schema.up.sql # SQL Schema Migration
│   ├── 000001_init_schema.down.sql # SQL Rollback
│   └── seeds.sql             # ข้อมูลตัวอย่างเริ่มต้น (Seed Data)
├── nginx/
│   ├── nginx.conf         # Nginx Configuration หลัก
│   └── conf.d/
│       └── default.conf    # Nginx Reverse Proxy & Header Rules
├── docs/
│   ├── API_SPEC.md        # API Specifications
│   ├── ER_DIAGRAM.md      # Database Documentation
│   ├── er_diagram.png     # ภาพ ER Diagram ความละเอียดสูง (2352x1893)
│   └── postman_collection.json # Postman Collection สำหรับทดสอบ
├── Dockerfile              # Multi-stage Docker build สำหรับ Go Service
├── docker-compose.yml      # ไฟล์ Compose รวม Nginx, Go Service,
PostgreSQL
├── Makefile                # คำสั่งหลักในการรันและทดสอบ
├── .env.example
├── .gitignore
└── README.md

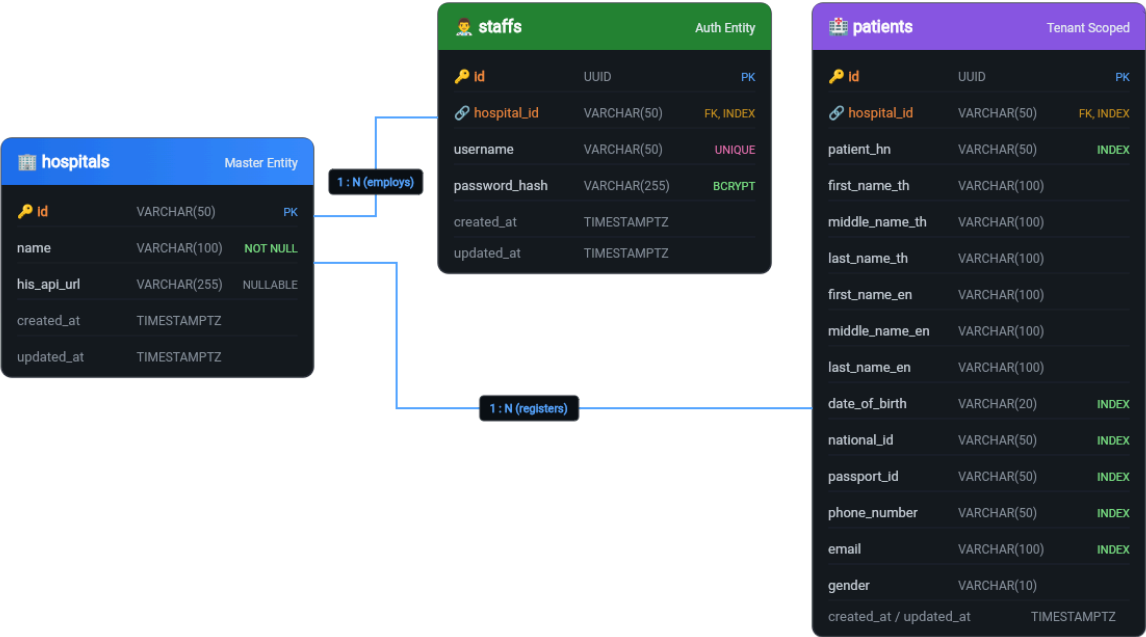
```

2. แผนภาพฐานข้อมูลและโครงสร้างข้อมูล (ER Diagram & Database Schema)

2.1 แผนภาพ Entity-Relationship (ER Diagram)

HOSPITAL MIDDLEWARE SYSTEM - ENTITY RELATIONSHIP DIAGRAM

Multi-Tenant Database Architecture with Hospital Isolation



คำอธิบายความสัมพันธ์ (Relationships):

- hospitals (1) → staffs (N): โรงพยาบาล 1 แห่ง มีเจ้าหน้าที่สังกัดได้หลายคน (1-to-Many)
- hospitals (1) → patients (N): โรงพยาบาล 1 แห่ง มีประวัติผู้ป่วยที่ขึ้นทะเบียนได้หลายคน (1-to-Many)
- Data Isolation Rule: เจ้าหน้าที่ (Staff) จะสามารถสืบค้นและเข้าถึงได้เฉพาะข้อมูลผู้ป่วยที่มี **hospital_id** ตรงกับโรงพยาบาลที่ตนเองสังกัดเท่านั้น

2.2 รายละเอียดตารางในฐานข้อมูล

ตาราง: hospitals (ข้อมูลโรงพยาบาล)

ชื่อคอลัมน์ (Column)	ชนิดข้อมูล (Type)	เงื่อนไข (Constraint)	คำอธิบาย

id	VARCHAR(50)	PRIMARY KEY	รหัสระบุโรงพยาบาล เช่น hospital-a, hospital-b
name	VARCHAR(100)	NOT NULL	ชื่อเต็มของโรงพยาบาล
his_api_url	VARCHAR(255)	NULLABLE	Base URL สำหรับเชื่อมต่อระบบ HIS ภายนอก
created_at	TIMESTAMPZ	DEFAULT NOW()	วันที่สร้างข้อมูล
updated_at	TIMESTAMPZ	DEFAULT NOW()	วันที่แก้ไขข้อมูลล่าสุด

ตาราง: staffs (ข้อมูลเจ้าหน้าที่)

ชื่อคอลัมน์ (Column)	ชนิดข้อมูล (Type)	เงื่อนไข (Constraint)	คำอธิบาย
id	UUID	PRIMARY KEY	รหัสประจำตัวเจ้าหน้าที่ (UUID v4)

username	VARCHAR(50)	UNIQUE, NOT NULL	ชื่อผู้ใช้งานสำหรับ Login (ไม่ซ้ำในระบบ)
password_hash	VARCHAR(255)	NOT NULL	รหัสผ่านที่เข้ารหัสด้วย Bcrypt Hash ป้องกันการรั่วไหล
hospital_id	VARCHAR(50)	NOT NULL, FK	Foreign Key อ้างอิง hospitals(id) ระบุโรงพยาบาลที่สังกัด
created_at	TIMESTAMPZ	DEFAULT NOW()	วันที่เวลาที่สร้างข้อมูล
updated_at	TIMESTAMPZ	DEFAULT NOW()	วันที่เวลาที่แก้ไขข้อมูลล่าสุด

ตาราง: patients (ข้อมูลประวัติผู้ป่วย)

ชื่อคอลัมน์ (Column)	ชนิดข้อมูล (Type)	เงื่อนไข (Constraint)	คำอธิบาย
id	UUID	PRIMARY KEY	รหัสประจำตัวผู้ป่วยภายในระบบ (UUID v4)
hospital_id	VARCHAR(50)	NOT NULL, FK	Foreign Key อ้างอิง hospitals(id) (ใช้แบ่งแยกสิทธิ์ข้อมูลตามโรงพยาบาล)

patient_hn	VARCHAR(50)	NOT NULL, INDEX	เลขประจำตัวผู้ป่วยของโรงพยาบาล (Hospital Number)
first_name_th	VARCHAR(100)	INDEX	ชื่อภาษาไทย
middle_name_th	VARCHAR(100)	NULLABLE	ชื่อกลางภาษาไทย
last_name_th	VARCHAR(100)	INDEX	นามสกุลภาษาไทย
first_name_en	VARCHAR(100)	INDEX	ชื่อภาษาอังกฤษ
middle_name_en	VARCHAR(100)	NULLABLE	ชื่อกลางภาษาอังกฤษ
last_name_en	VARCHAR(100)	INDEX	นามสกุลภาษาอังกฤษ
date_of_birth	VARCHAR(20)	INDEX	วันเดือนปีเกิดของผู้ป่วย (รูปแบบ: YYYY-MM-DD เช่น 1990-05-15)
national_id	VARCHAR(50)	INDEX	เลขประจำตัวประชาชน 13 หลัก
passport_id	VARCHAR(50)	INDEX	เลขหนังสือเดินทาง (Passport)
phone_number	VARCHAR(50)	INDEX	เบอร์โทรศัพท์ติดต่อ
email	VARCHAR(100)	INDEX	อีเมลของผู้ป่วย

gender	VARCHAR(10)	NULLABLE	เพศของผู้ป่วย (M = ชาย, F = หญิง, Other)
created_at	TIMESTAMPZ	DEFAULT NOW()	วันที่สร้างข้อมูล
updated_at	TIMESTAMPZ	DEFAULT NOW()	วันที่แก้ไขข้อมูลล่าสุด

2.3 กลยุทธ์การสร้างดัชนี (Indexing & Multi-Tenancy Strategy)

เพื่อรองรับการค้นหาที่รวดเร็วและปลอดภัย ฐานข้อมูลจึงสร้าง Composite Index โดยมี **hospital_id** เป็น Prefix นำหน้าในทุกเงื่อนไขการค้นหา:

- `idx_patients_hospital_id` บน `patients(hospital_id)`
 - `idx_patients_hospital_hn` บน `patients(hospital_id, patient_hn)`
 - `idx_patients_hospital_national_id` บน `patients(hospital_id, national_id)`
 - `idx_patients_hospital_passport_id` บน `patients(hospital_id, passport_id)`
 - `idx_patients_hospital_phone` บน `patients(hospital_id, phone_number)`
 - `idx_patients_hospital_email` บน `patients(hospital_id, email)`
 - `idx_patients_hospital_dob` บน `patients(hospital_id, date_of_birth)`
 - `idx_patients_hospital_name_th` บน `patients(hospital_id, first_name_th, last_name_th)`
 - `idx_patients_hospital_name_en` บน `patients(hospital_id, first_name_en, last_name_en)`
-

3. ข้อกำหนด API (API Specifications)

3.1 ตรวจสอบสถานะระบบ (Health Check)

GET /health

คำอธิบาย: ตรวจสอบสถานะการทำงานของ Web Server และการเชื่อมต่อกับฐานข้อมูล PostgreSQL

ผลลัพธ์ (Response - 200 OK):

```
{
  "success": true,
  "message": "Service is healthy",
  "data": {
    "database": "up",
    "service": "hospital-middleware-api",
    "status": "healthy"
  }
}
```

3.2 สร้างบัญชีเจ้าหน้าที่ (Create Hospital Staff)

POST /staff/create

คำอธิบาย: ลงทะเบียนเจ้าหน้าที่โรงพยาบาลใหม่ พร้อมกำหนดรหัสผ่าน (ระบบจะเข้ารหัสด้วย Bcrypt) และระบุโรงพยาบาลที่สังกัด

Request Body:

Field	Type	Required	คำอธิบาย
username	string	Yes	ชื่อผู้ใช้งานสำหรับ Login (ขั้นต่ำ 3 ตัวอักษร, ไม่ซ้ำในระบบ)
password	string	Yes	รหัสผ่าน (ขั้นต่ำ 6 ตัวอักษร)
hospital	string	Yes	รหัสโรงพยาบาลที่สังกัด เช่น hospital-a, hospital-b

ตัวอย่าง Request Body:

```
{
  "username": "doctor_somchai",
  "password": "Password123!",
  "hospital": "hospital-a"
}
```

ผลลัพธ์สำเร็จ (Response - 201 Created):

```
{
  "success": true,
  "message": "Staff created successfully",
  "data": {
    "id": "e81d432e-b4a8-4261-9f44-88ba42f5ee99",
    "username": "doctor_somchai",
    "hospital": "hospital-a",
    "created_at": "2026-08-28T09:00:00Z"
  }
}
```

3.3 เข้าสู่ระบบเจ้าหน้าที่ (Staff Login)

POST /staff/login

คำอธิบาย: ยืนยันตัวตนเจ้าหน้าที่ด้วย username, password และ hospital เพื่อรับ JWT Bearer Token (อายุ 24 ชั่วโมง)

Request Body:

Field	Type	Required	คำอธิบาย
username	string	Yes	ชื่อผู้ใช้งาน
password	string	Yes	รหัสผ่าน
hospital	string	Yes	รหัสโรงพยาบาลที่สังกัด (ต้องตรงกับข้อมูลในระบบ)

ตัวอย่าง Request Body:

```
{
  "username": "doctor_somchai",
  "password": "Password123!",
  "hospital": "hospital-a"
}
```

ผลลัพธ์สำเร็จ (Response - 200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 86400,
    "staff": {
      "id": "e81d432e-b4a8-4261-9f44-88ba42f5ee99",
      "username": "doctor_somchai",
      "hospital": "hospital-a",
      "created_at": "2026-08-28T09:00:00Z"
    }
  }
}
```

3.4 ค้นหาข้อมูลผู้ป่วย (Patient Search)

GET /patient/search POST /patient/search Requires Login (JWT)

คำอธิบาย: ค้นหาข้อมูลผู้ป่วยตามเงื่อนไขที่กำหนด โดยระบบจะ จำกัดขอบเขตการค้นหาเฉพาะผู้ป่วยที่อยู่ในโรงพยาบาลเดียวกับเจ้าหน้าที่ Login เสมอ (Multi-Tenancy Data Isolation)

 **ระบบความปลอดภัยและการแยกข้อมูล (Tenant Isolation):** ระบบจะอ่านข้อมูล **hospital** จาก JWT Token ที่ผ่านการถอดรหัสแล้วโดยตรง เจ้าหน้าที่ของ Hospital A จะไม่มีสิทธิ์เข้าถึงหรือเห็นข้อมูลผู้ป่วยของ Hospital B ในทุกกรณี

Request Headers:

Header	Value	Required	คำอธิบาย
Authorization	Bearer <JWT_TOKEN>	Yes	Token ที่ได้จากการ Login สำเร็จ

Query Parameters (ทุกฟิลด์เป็น Optional):

Parameter	Type	Required	คำอธิบาย
national_id	string	No	เลขประจำตัวประชาชน 13 หลัก
passport_id	string	No	เลขหนังสือเดินทาง (Passport)
first_name	string	No	ชื่อผู้ป่วย (ค้นหาได้ทั้งภาษาไทยและภาษาอังกฤษแบบ Partial Match)
middle_name	string	No	ชื่อกลาง (ภาษาไทยหรืออังกฤษ)

last_name	string	No	นามสกุล (ภาษาไทยหรืออังกฤษ)
date_of_birth	string	No	วันเดือนปีเกิดของผู้ป่วย (รูปแบบ: YYYY-MM-DD เช่น 1990-05-15)
phone_number	string	No	เบอร์โทรศัพท์ติดต่อ
email	string	No	อีเมลของผู้ป่วย

ตัวอย่างการเรียกใช้งาน (Examples):

- ค้นหาตามชื่อ: `GET /patient/search?first_name=Somchai`
- ค้นหาตามเลขบัตรประชาชน: `GET /patient/search?national_id=1103701234567`
- ค้นหาตามวันเกิดและเบอร์โทร: `GET /patient/search?date_of_birth=1990-05-15&phone_number=0812345678`
- ค้นหาทั้งหมดในโรงพยาบาล: `GET /patient/search` (ไม่ส่ง parameter)

ผลลัพธ์สำเร็จ (Response - 200 OK):

```
{
  "success": true,
  "message": "Patients retrieved successfully",
  "count": 1,
  "data": [
    {
      "id": "c3333333-3333-3333-3333-333333333333",
      "hospital_id": "hospital-a",
      "patient_hn": "HN-A-1001",
      "first_name_th": "สมชาย",
      "middle_name_th": "",
      "last_name_th": "ใจดี",
      "first_name_en": "Somchai",
      "middle_name_en": "",
      "last_name_en": "Jaidee",
      "date_of_birth": "1990-05-15",
      "national_id": "1103701234567",
      "passport_id": "AA123456",
      "phone_number": "0812345678",
      "email": "somchai@example.com",
      "gender": "M",
      "created_at": "2026-08-28T09:00:00Z"
    }
  ]
}
```

```
}  
]  
}
```